2026 ENTERPRISE EDITION

CLOUD DATA PLATFORMS

DATTA SABLE TECHNICAL LIBRARY

Snowflake Cloud Data Platform Architecture & FinOps

The definitive production guide to multi-cluster warehouse sizing, micro-partition pruning, continuous Snowpipe streaming, Zero-Copy Cloning, and aggressive FinOps credit governance.

22
Production Chapters & Architecture Blueprints

100%
Practical Code, Schemas & Frameworks

2026
Modern Enterprise Best Practices

INDEPENDENTLY AUTHORED BY

Datta Sable
Enterprise BI & Data Architect

Published: October 2026
dattasable.com/ebooks

ARCHITECTURE BLUEPRINT

Table of Contents & Snowflake Platform Blueprint

EXECUTIVE OVERVIEW

Snowflake revolutionised data warehousing by completely decoupling storage, compute, and cloud services. Operating Snowflake at scale requires orchestrating auto-scaling virtual warehouses, continuous data pipelines, and implementing disciplined FinOps governance to prevent runaway credit consumption.



Comprehensive Chapter Guide

Part I: Engine & Storage

- Ch 3: Virtual Warehouse Sizing
- Ch 4: Micro-Partitions & Pruning
- Ch 5: Cloud Services Layer
- Ch 6: Snowpipe & Streaming
- Ch 7: Streams & Task Pipelines

Part II: Data Management

- Ch 8: Zero-Copy Cloning Sandboxes
- Ch 9: Time Travel & Fail-safe
- Ch 10: Semi-Structured JSON/VARIANT
- Ch 11: FinOps Credit Governance
- Ch 12: Account Usage Analytics

Part III: Enterprise Security & Governance: Ch 13 Snowpark & Python, Ch 14 Secure Data Sharing, Ch 15 RBAC Role Hierarchies, Ch 16 Dynamic Data Masking & Row Access, Ch 17 Cross-Cloud Disaster Recovery, Ch 18 Query Profile Deep Dive, Ch 19 E-Commerce FinOps Case Study, Ch 20 Cost Traps, Ch 21 SQL Diagnostic Scripts, Ch 22 SnowPro Architect Audit Checklist.

CHAPTER 01

Virtual Warehouses: Sizing & Multi-Cluster Scaling

COMPUTE ARCHITECTURE

A Virtual Warehouse is an MPP compute cluster allocated on demand. Warehouses do not store persistent data; they execute SQL queries, transform data, and populate local SSD caches.

Warehouse T-Shirt Sizes & Credit Burn Rate

Each size increase exactly doubles compute capacity and credit burn per operating hour:

T-Shirt Size	Compute Nodes	Credits / Hour	Ideal Production Workload
X-Small	1 node	1 credit	Staging ingestion, lightweight dbt tests, ad-hoc queries
Small	2 nodes	2 credits	Standard departmental reporting, single-table aggregations
Medium	4 nodes	4 credits	Daily batch dbt transformation runs, multi-table joins
Large / X-Large	8 – 16 nodes	8 – 16 credits	High-volume historical backfills, 500M+ row fact builds

Multi-Cluster Auto-Scaling

Scaling 'Up' (larger T-shirt size) speeds up large individual queries. Scaling 'Out' (adding clusters to a multi-cluster warehouse) handles concurrency spikes without query queuing.

create_multi_cluster_wh.sql

```sql
CREATE WAREHOUSE bi_reporting_wh
  WITH WAREHOUSE_SIZE = 'MEDIUM'
  MIN_CLUSTER_COUNT = 1
  MAX_CLUSTER_COUNT = 5
  SCALING_POLICY = 'STANDARD'
  AUTO_SUSPEND = 60           -- Suspend after 60 seconds of idle time
  AUTO_RESUME = TRUE
  INITIALLY_SUSPENDED = TRUE;
```

CHAPTER 02

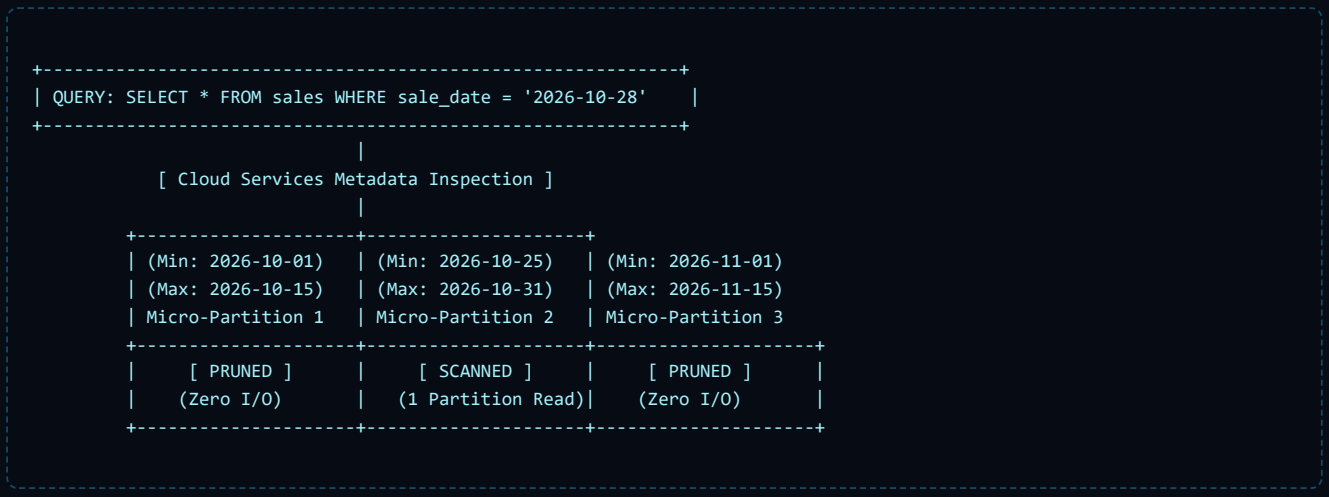
Storage Layer Internals: Micro-Partitions & Pruning

STORAGE INTERNALS

Snowflake stores all data in immutable, compressed, columnar files called **Micro-Partitions**. Each micro-partition contains between 50MB and 500MB of uncompressed data.

How Metadata Drives Micro-Partition Pruning

During data ingestion, the Cloud Services Layer automatically extracts metadata for each micro-partition: min/max values for every column, null counts, and distinct value distributions. When a query executes with a filter, Snowflake consults metadata and skips unneeded micro-partitions entirely without reading them from storage.



Explicit Clustering Keys

For tables > 1TB where queries filter on columns other than the natural ingestion sequence, define a clustering key:

cluster_key_ddl.sql

```
ALTER TABLE core.sales.fact_transactions
CLUSTER BY (transaction_date, region_id);
```

CHAPTER 03

The Cloud Services Layer: Optimization & Billing

THE BRAIN OF SNOWFLAKE

The Cloud Services Layer ties the platform together: authenticating connections, parsing and optimizing SQL queries, managing ACID transactions, and tracking metadata security policies.

Free vs. Billable Cloud Services

Snowflake provides free Cloud Services compute up to **10% of total daily Virtual Warehouse spend**. If an enterprise consumes 100 warehouse credits in a day, up to 10 Cloud Services credits are completely waived.

Operation Type	Layer Consumed	Cost Profile
`SELECT COUNT(*) FROM table`	Cloud Services Only	Zero warehouse credits; resolved instantly from metadata
Zero-Copy Clone	Cloud Services Only	Zero storage or warehouse cost; updates metadata pointers
Query Compilation & Optimization	Cloud Services	Waived within 10% daily credit allowance
High-frequency `SHOW TABLES` polling	Cloud Services Only	Can trigger billable overage if polling exceeds 10% allowance

FINOPS ALERT

BI tools or third-party orchestration tools that issue `SHOW OBJECTS` every 5 seconds can inflate Cloud Services consumption beyond the 10% threshold, generating surprise bills.

CHAPTER 04

Continuous Ingestion: Snowpipe & Snowpipe Streaming

REAL-TIME INGESTION

Snowpipe is Snowflake's serverless continuous ingestion service. It listens for cloud storage notifications (S3 SQS, Azure Event Grid) and loads newly arrived micro-batches automatically without user warehouse compute.

Configuring an Automated Snowpipe

create_snowpipe.sql

```sql
CREATE OR REPLACE PIPE raw.ingestion.pipe_web_clicks
  AUTO_INGEST = TRUE
  INTEGRATION = 'AZURE_STORAGE_NOTIFICATION'
AS
COPY INTO raw.events.web_clicks
FROM @raw.stages.stage_azure_clicks
FILE_FORMAT = (TYPE = 'JSON')
ON_ERROR = 'CONTINUE';
```

Snowpipe vs. Snowpipe Streaming

Capability	Traditional Snowpipe	Snowpipe Streaming (SDK)
Data Transit	Writes files to S3/Blob first	Direct streaming from Kafka/Java SDK
Ingestion Latency	30 seconds – 2 minutes	Sub-second (< 1 second)
Cost Model	Per-file serverless compute fee	Direct micro-partition row ingestion

CHAPTER 05

Continuous Data Pipelines: Streams & Tasks

SERVERLESS CDC

Snowflake Streams record Change Data Capture (CDC) deltas against source tables, while Tasks execute scheduled SQL transformations, creating automated ELT pipelines natively within Snowflake.

Creating an Append-Only Stream

streams_and_tasks.sql

```sql
-- 1. Create CDC stream tracking new insertions
CREATE OR REPLACE STREAM raw.events.stream_raw_orders
ON TABLE raw.events.orders
APPEND_ONLY = TRUE;

-- 2. Create serverless scheduled task that executes only when data is pending
CREATE OR REPLACE TASK core.tasks.task_process_orders
  USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE = 'XSMALL'
  SCHEDULE = '5 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('raw.events.stream_raw_orders')
AS
INSERT INTO core.sales.fact_orders (order_id, customer_id, amount, processed_at)
SELECT order_id, customer_id, amount, CURRENT_TIMESTAMP()
FROM raw.events.stream_raw_orders;

-- 3. Resume the newly created task
ALTER TASK core.tasks.task_process_orders RESUME;
```

Why SYSTEM$STREAM_HAS_DATA Saves Thousands

Without the `WHEN` condition, a task scheduled every 5 minutes spins up warehouse compute 288 times a day even when zero rows arrive. `SYSTEM$STREAM_HAS_DATA` checks stream metadata in Cloud Services for free, firing warehouse compute only when real data exists.

CHAPTER 06

Zero-Copy Cloning: Sandboxes & Dev Environments

METADATA-ONLY CLONING

Zero-Copy Cloning creates instant copies of databases, schemas, or tables without duplicating physical storage micro-partitions. Clones are fully independent, read/write sandboxes.

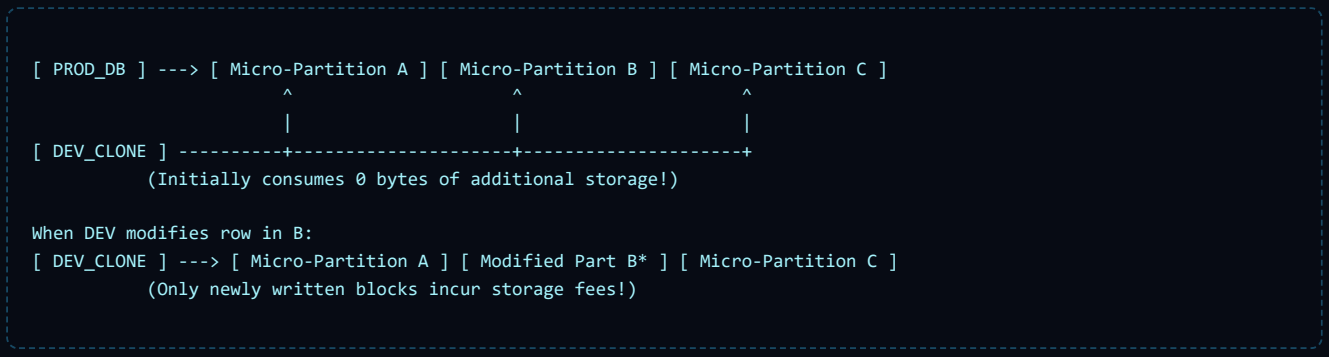
Instant Multi-Terabyte Production Clones

zero_copy_cloning.sql

```sql
-- Clone a 10TB production database in 3 seconds for staging tests
CREATE DATABASE staging_sandbox
CLONE production_db;

-- Clone table at an exact historical timestamp (Time Travel + Clone)
CREATE TABLE dev.finance.fact_revenue_clone
CLONE prod.finance.fact_revenue
AT (TIMESTAMP => '2026-10-28 10:00:00 -0700'::timestamp_tz);
```

Copy-on-Write Storage Economics



CHAPTER 07

Time Travel & Fail-Safe: Disaster Protection

DATA RECOVERY

Snowflake's immutable micro-partition architecture provides native point-in-time historical querying, table restoration, and catastrophic disaster recovery through Time Travel and Fail-safe.

Time Travel Recovery Mechanics

Standard tables support up to 90 days of Time Travel (Enterprise Edition). If an engineer accidentally runs `DROP TABLE customers;` or corrupts data with an unconstrained UPDATE, the table can be restored in seconds:

time_travel_restoration.sql

```sql
-- 1. Restore an accidentally dropped table instantly
UNDROP TABLE core.crm.customers;

-- 2. Query table as it existed before a rogue transaction executed
SELECT * FROM core.crm.customers
BEFORE (STATEMENT => '01b34a90-0001-44ab-0002-89cd00034a12');
```

Fail-Safe: The 7-Day Safety Net

Once Time Travel expires, data enters a non-configurable 7-day **Fail-safe** window. Fail-safe is accessible only by Snowflake Support for disaster recovery (e.g. recovering from regional storage cataclysms). Customers pay standard storage fees for Fail-safe historical blocks.

STORAGE OPTIMIZATION

Use `TRANSIENT` or `TEMPORARY` tables for ETL staging layers. Transient tables have 0 or 1 day of Time Travel and ZERO Fail-safe, slashing intermediate storage costs by 80%.

CHAPTER 08

Semi-Structured Data: VARIANT, OBJECT & FLATTEN

POLYMORPHIC JSON MASTERY

Snowflake treats semi-structured data (JSON, Avro, Parquet, XML) as a first-class citizen using the native `VARIANT` datatype. The engine automatically shreds nested JSON elements into columnar micro-partitions under the hood.

Dot-Notation & Path Traversal

query_json_variant.sql

```sql
-- Extracting scalar values and casting types from VARIANT column
SELECT
    raw_payload:order_id::INT AS order_id,
    raw_payload:customer.email::VARCHAR AS customer_email,
    raw_payload:meta.tags[0]::VARCHAR AS primary_tag
FROM raw.events.orders_stream;
```

Exploding Arrays with the FLATTEN Table Function

When JSON payloads contain nested arrays (e.g. line items within an invoice), the `FLATTEN` function explodes array elements into relational rows:

flatten_json_array.sql

```sql
SELECT
    o.raw_payload:order_id::INT AS order_id,
    item.value:item_sku::VARCHAR AS item_sku,
    item.value:quantity::INT AS quantity,
    item.value:unit_price::DECIMAL(10,2) AS unit_price
FROM raw.events.orders_stream o,
LATERAL FLATTEN(input => o.raw_payload:line_items) item;
```

CHAPTER 09

Snowflake FinOps: Credit Quotas & Governance

FINANCIAL GOVERNANCE

Without automated guardrails, an accidental infinite Cartesian join running on an X-Large warehouse will burn hundreds of dollars before morning. Resource Monitors provide automated circuit breakers.

Deploying a Tiered Resource Monitor

resource_monitor_ddl.sql

```
CREATE OR REPLACE RESOURCE MONITOR rm_analytics_monthly
  WITH CREDIT_QUOTA = 2000
  FREQUENCY = 'MONTHLY'
  START_TIMESTAMP = 'IMMEDIATELY'
  TRIGGERS
    ON 75 PERCENT DO NOTIFY
    ON 90 PERCENT DO NOTIFY
    ON 100 PERCENT DO SUSPEND              -- Let active queries finish, block new ones
    ON 110 PERCENT DO SUSPEND_IMMEDIATELY;  -- Kill all active queries immediately

ALTER WAREHOUSE bi_reporting_wh
SET RESOURCE_MONITOR = rm_analytics_monthly;
```

Query Timeout Safety Bounds

Set `STATEMENT_TIMEOUT_IN_SECONDS = 1800` (30 minutes) at the warehouse level. If a rogue query hangs or blocks on table locks, Snowflake terminates the query after 30 minutes, preventing 24-hour idle runaway charges.

CHAPTER 10

Account Usage Views & Cost Diagnostics

OBSERVABILITY

The SNOWFLAKE.ACCOUNT_USAGE schema provides 365 days of historical audit telemetry, query execution profiles, and billing breakdowns queryable directly with SQL.

Top 5 Most Expensive Queries in the Past 30 Days

top_expensive_queries.sql

```sql
SELECT
    query_id,
    user_name,
    warehouse_name,
    execution_time / 1000 AS execution_seconds,
    SUBSTRING(query_text, 1, 80) AS truncated_query,
    bytes_scanned / 1024 / 1024 / 1024 AS gb_scanned
FROM snowflake.account_usage.query_history
WHERE start_time >= DATEADD('day', -30, CURRENT_TIMESTAMP())
ORDER BY execution_time DESC
LIMIT 5;
```

Warehouse Metering by Department

warehouse_spend_breakdown.sql

```sql
SELECT
    warehouse_name,
    DATE_TRUNC('month', start_time) AS billing_month,
    SUM(credits_used) AS total_credits,
    ROUND(SUM(credits_used) * 3.00, 2) AS estimated_cost_usd
FROM snowflake.account_usage.warehouse_metering_history
WHERE start_time >= DATEADD('month', -3, CURRENT_TIMESTAMP())
GROUP BY ALL
ORDER BY total_credits DESC;
```

CHAPTER 11

Snowpark Python & Stored Procedures

NATIVE PYTHON EXECUTION

Snowpark executes Python, Java, and Scala code securely within Snowflake's virtual warehouse sandbox. DataFrames compile into lazy SQL expressions, executing data transformations directly in the engine without data extraction.

Building a Snowpark Data Pipeline

snowpark_pipeline.py

```python
from snowflake.snowpark import Session
from snowflake.snowpark.functions import col, when, current_timestamp

def cleanse_orders(session: Session) -> str:
    # Lazy DataFrame definition - zero data moved to local client!
    df = session.table("raw.sales.orders")

    df_transformed = df.filter(col("order_status") == "COMPLETED") \
        .with_column("net_usd", col("amount") * 0.95) \
        .with_column("processed_at", current_timestamp())

    # Writes directly to target Snowflake table in warehouse memory
    df_transformed.write.mode("append").save_as_table("core.sales.orders_clean")
    return "Successfully cleansed orders"
```

Deploying as an In-Engine Stored Procedure

register_procedure.sql

```sql
CREATE OR REPLACE PROCEDURE core.sales.sp_cleanse_orders()
  RETURNS STRING
  LANGUAGE PYTHON
  RUNTIME_VERSION = '3.10'
  PACKAGES = ('snowflake-snowpark-python')
  HANDLER = 'cleanse_orders'
  AS $$
# Inline python function definition
$$;
```

CHAPTER 12

Secure Data Sharing & Data Clean Rooms

ZERO DATA MOVEMENT

Traditional data sharing relies on brittle SFTP exports or REST APIs that duplicate data and leak control. Snowflake Secure Data Sharing grants external consumers direct read access to live data without copying files.

Creating a Secure Share

secure_share_producer.sql

```
-- Producer: Create share and grant read permissions
CREATE SHARE vendor_partner_share;
GRANT USAGE ON DATABASE core_analytics TO SHARE vendor_partner_share;
GRANT USAGE ON SCHEMA core_analytics.partner_feeds TO SHARE vendor_partner_share;
GRANT SELECT ON TABLE core_analytics.partner_feeds.daily_inventory TO SHARE vendor_partner_share;

-- Add consumer Snowflake account locator
ALTER SHARE vendor_partner_share ADD ACCOUNTS = xy12345.eu-west-1;
```

Consumer Experience

The consumer account executes: `CREATE DATABASE partner_inventory FROM SHARE myorg.account.vendor_partner_share;`. The database is instantly queryable. The producer pays for storage; the consumer pays for their own query compute.

CHAPTER 13

RBAC: System Roles & Functional Hierarchies

SECURITY GOVERNANCE

Snowflake enforces a strict, hierarchical Role-Based Access Control (RBAC) model. Designing functional and access role hierarchies prevents permission chaos and privilege escalation.



The Functional vs Access Role Rule

- **Access Roles (AR):** Granted direct permissions on database objects (e.g. `GRANT SELECT ON ALL TABLES IN SCHEMA sales TO ROLE ar_sales_read`). Never assigned directly to users.
- **Functional Roles (FR):** Model job functions (e.g. `bi_developer`, `finance_analyst`). Inherit multiple Access Roles and are granted directly to end-users.

CHAPTER 14

Dynamic Data Masking & Row Access Policies

GRANULAR SECURITY

Dynamic Data Masking and Row Access Policies (RAP) evaluate user roles at query runtime, masking sensitive PII columns or filtering rows on the fly without duplicating underlying physical tables.

Dynamic Masking Policy: Hashing Emails

masking_policy.sql

```sql
CREATE OR REPLACE MASKING POLICY core.security.mask_email AS (val VARCHAR)
RETURNS VARCHAR ->
  CASE
    WHEN CURRENT_ROLE() IN ('COMPLIANCE_OFFICER', 'ACCOUNTADMIN') THEN val
    ELSE REGEXP_REPLACE(val, '(^.).*(@.*$)', '\\1***\\2') -- e.g. j***@company.com
  END;

-- Apply policy to column
ALTER TABLE core.crm.customers
MODIFY COLUMN email SET MASKING POLICY core.security.mask_email;
```

Row Access Policy: Multi-Tenant Region Isolation

row_access_policy.sql

```sql
CREATE OR REPLACE ROW ACCESS POLICY core.security.rap_region AS (region_code VARCHAR)
RETURNS BOOLEAN ->
  CURRENT_ROLE() = 'GLOBAL_ADMIN' OR
  region_code = CURRENT_USER();
```

CHAPTER 15

Disaster Recovery: Cross-Cloud & Region Replication

BUSINESS CONTINUITY

Snowflake's cloud-agnostic architecture enables continuous database replication across cloud providers (AWS, Azure, GCP) and geographical regions, guaranteeing failover survival during major public cloud outages.

Configuring Account Replication

cross_cloud_replication.sql

```sql
-- In Primary Account (AWS us-east-1)
ALTER DATABASE core_sales_db
ENABLE REPLICATION TO ACCOUNTS myorg.azure_westeurope_account;

-- In Secondary Account (Azure westeurope)
CREATE DATABASE core_sales_db
AS REPLICA OF myorg.aws_useast1_account.core_sales_db;

-- Execute initial synchronization refresh
ALTER DATABASE core_sales_db REFRESH;
```

Failover & RTO/RPO Metrics

- **RTO (Recovery Time Objective):** < 5 minutes. Execute `ALTER DATABASE core_sales_db PRIMARY;` to promote secondary database instantly.
- **RPO (Recovery Point Objective):** Bounded by replication refresh schedules (e.g. 15 minutes).

CHAPTER 16

Query Profile Analysis: Exploding Joins & Spills

DIAGNOSTIC MASTERY

Snowflake's visual Query Profile breaks down physical execution metrics: percentage of time spent in processing, local disk spilling, remote storage spilling, and micro-partition pruning efficiency.

Key Performance Red Flags in Query Profiles

Profile Metric Indicator	Underlying Architecture Bottleneck	Remediation Action
`Spilling to local storage: 120GB`	Data for hash joins or aggregations exceeds virtual warehouse RAM	Increase warehouse T-shirt size (Medium -> Large)
`Spilling to remote storage: 45GB`	Local SSD cache exhausted; writing temporary blocks to S3/Blob! Severe 10x penalty!	Immediate resize to larger warehouse; refactor exploding joins
`Partitions scanned: 4,000 / 4,000 (0% pruned)`	Query filters on unclustered columns; full table scan executed	Define clustering key or filter on ingestion timestamp
`Join operator outputs 50x input rows`	Cartesian product or non-unique join keys exploding cardinality	Deduplicate dimension keys; review join predicates

REMOTE SPILL HEURISTIC

If a query spills to *remote* storage, resizing the warehouse to double its size usually cuts execution time by 75% while burning virtually the same total net credits.

CHAPTER 17

Production FinOps Case Study: $1.2M Cost Optimization

ENTERPRISE CASE STUDY

A global retail e-commerce conglomerate saw annual Snowflake consumption surge from $400K to $1.8M. Implementing structured FinOps governance, auto-suspend tuning, and workload segregation reduced annual spend to $600K without degrading SLAs.

Phase 1: Workload Segregation

Discovered that 80 reporting analysts and 12 automated ETL ingestion tasks shared a single 24/7 running `X-LARGE` warehouse. Split into:

- `ETL_WH` (Large, auto-suspend 60s, runs only during pipeline windows)
- `BI_REPORTING_WH` (Medium multi-cluster 1-3, auto-suspend 60s)
- `AD_HOC_WH` (Small, statement timeout 10 minutes)

Phase 2: Auto-Suspend Optimization

Default auto-suspend on 8 warehouses was set to 10 minutes (600s). Because Snowflake charges in 1-second increments after the initial 60 seconds, reducing auto-suspend to 60 seconds across all warehouses eliminated 400 idle credit hours monthly.

Phase 3: Transient Staging Tables

Converted 50TB of ephemeral staging tables to `TRANSIENT`, eliminating 7-day Fail-safe storage fees and saving $14,000 monthly on cloud storage invoices.

CHAPTER 18

Common Snowflake FinOps Traps & How to Avoid Them

PITFALL PREVENTION

Avoid these 4 ubiquitous operational configuration mistakes that bleed enterprise budgets in Snowflake environments.

FinOps Trap	Why It Wastes Money	Enforced Standard
Auto-Suspend > 5 Minutes	Warehouses sit idle burning credits long after analysts close their dashboards	Set `AUTO_SUSPEND = 60` on all interactive reporting warehouses
Over-Clustering Small Tables	Continuous Background Re-clustering burns serverless credits on tables < 500GB	Never cluster tables < 250GB. Natural micro-partitioning is sufficient
Single Massive Shared Warehouse	A 10-second query keeps an X-Large warehouse alive for 60 seconds (16 credits/hr)	Segregate workloads by warehouse size and operational domain
Infinite Time Travel on Stage Tables	Paying 90 days of storage + 7 days of Fail-safe on disposable intermediate dumps	Set `DATA_RETENTION_TIME_IN_DAYS = 0` on staging schemas

The Auto-Resume Ping Storm

BI tools querying Snowflake periodically to test connectivity (e.g. heartbeat pings) will wake up suspended warehouses every 3 minutes. Configure dedicated lightweight X-Small warehouses for connection polling.

CHAPTER 19

Snowflake FinOps Diagnostic Queries & Dashboards

DIAGNOSTIC RUNBOOK

Deploy these production SQL queries into your enterprise monitoring dashboards to track credit spikes and identify underutilized compute clusters.

finops_diagnostic_queries.sql

```sql
-- 1. Identify warehouses with highest idle credit waste
SELECT
    warehouse_name,
    SUM(credits_used) AS total_credits,
    SUM(credits_used_compute) AS compute_credits,
    SUM(credits_used_cloud_services) AS cloud_service_credits
FROM snowflake.account_usage.warehouse_metering_history
WHERE start_time >= DATEADD('day', -7, CURRENT_TIMESTAMP())
GROUP BY warehouse_name
ORDER BY total_credits DESC;

-- 2. Detect queries that spilled to remote disk in the last 24 hours
SELECT query_id, user_name, warehouse_name, warehouse_size,
       bytes_spilled_to_remote_storage / 1024 / 1024 / 1024 AS remote_spill_gb,
       execution_time / 1000 AS execution_sec
FROM snowflake.account_usage.query_history
WHERE start_time >= DATEADD('hour', -24, CURRENT_TIMESTAMP())
AND bytes_spilled_to_remote_storage > 0
ORDER BY remote_spill_gb DESC
LIMIT 10;
```

CHAPTER 20

SnowPro Enterprise Architect Production Audit

PRE-FLIGHT ASSESSMENT

Verify every architectural standard across this checklist before certifying any Snowflake enterprise account for production workloads.

- ☐ **Warehouse Boundaries:** Workloads segregated across dedicated warehouses (ETL, BI, Ad-Hoc); all warehouses configured with `AUTO_SUSPEND ≤ 60`.
- ☐ **Resource Monitors:** Monthly credit quotas assigned to every warehouse with automatic suspend circuit breakers at 100% threshold.
- ☐ **Storage Optimization:** Transient tables utilized for raw ETL staging; permanent tables configured with appropriate Time Travel retention policies.
- ☐ **Continuous Ingestion:** Snowpipe pipelines configured with serverless notifications; streams and tasks protected with `SYSTEM$STREAM_HAS_DATA` guards.
- ☐ **RBAC Discipline:** Permissions granted via Functional & Access Role hierarchy; MFA enforced on all human users; ACCOUNTADMIN access audited.
- ☐ **Data Protection:** Dynamic Data Masking applied to all PII attributes; cross-region disaster recovery replication verified.

AUTHOR CLOSING STATEMENT

Snowflake provides unmatched analytical agility when coupled with disciplined engineering governance. By decoupling compute workloads, automating ingestion pipelines, and enforcing strict FinOps boundaries, your organization scales analytical power while protecting financial margins.

Datta Sable · Enterprise Analytics & Data Architect · *dattasable.com*